1. **Project Structure:** The following output shows the main structure of my Assignment 6 project:

```
assignment06-weather-advisor:
README.md
advisor-app
create_database.py
create_weather_stack.py
delete_database.py
delete_weather_stack.py
grant_lab_rds_access.py
grant_lab_ssm_access.py
pytest.ini
requirements.txt
stack
store_gemini_key.py
tests
weather-app

assignment06-weather-advisor/advisor-app:
app.js
package-lock.json
package.json

assignment06-weather-advisor/stack:
__init__.py
balancer.py
clients.py
compute.py
config.py
database.py
network.py
outputs.py
policies.py
runner.py
scaling.py
secrets.py
storage.py
util.py

assignment06-weather-advisor/tests:
```

__init__.py
conftest.py
helpers.py
integration
unit

assignment06-weather-advisor/tests/integration:
__init__.py
test_application.py
test_infrastructure.py
test_recovery.py

I organized the project into separate Weather and Advisor applications, reusable Python infrastructure modules, database and stack creation/deletion scripts, and automated tests. This structure keeps the application code, AWS infrastructure code, and testing components separated and easier to manage.

2. **National Weather Service API Test:** I tested the National Weather Service API before deploying the full application. The first request returned the forecast metadata for the requested location, and the second request returned the actual weather forecast. Two upstream calls are required because the first call identifies the correct forecast endpoint for the location, while the second call retrieves the forecast data itself.

3. Database Configuration: I created the PostgreSQL database in Amazon RDS using the required low-cost and secure configuration. The final database settings were:

    a. Instance class: db.t4g.micro
    b. Engine: postgres
    c. Multi-AZ: False
    d. Publicly accessible: False
    e. Storage encrypted: True
    f. Backup retention: 0 days
    g. DB snapshots: []
    h. Automated backups: []

    This configuration kept the database private and encrypted while avoiding retained backups or snapshots for the temporary lab environment.

    Class: db.t4g.micro

    Engine: postgres

    MultiAZ: False

Public: False

Encrypted: True

BackupRetentionDays: 0

DB Snapshots: []

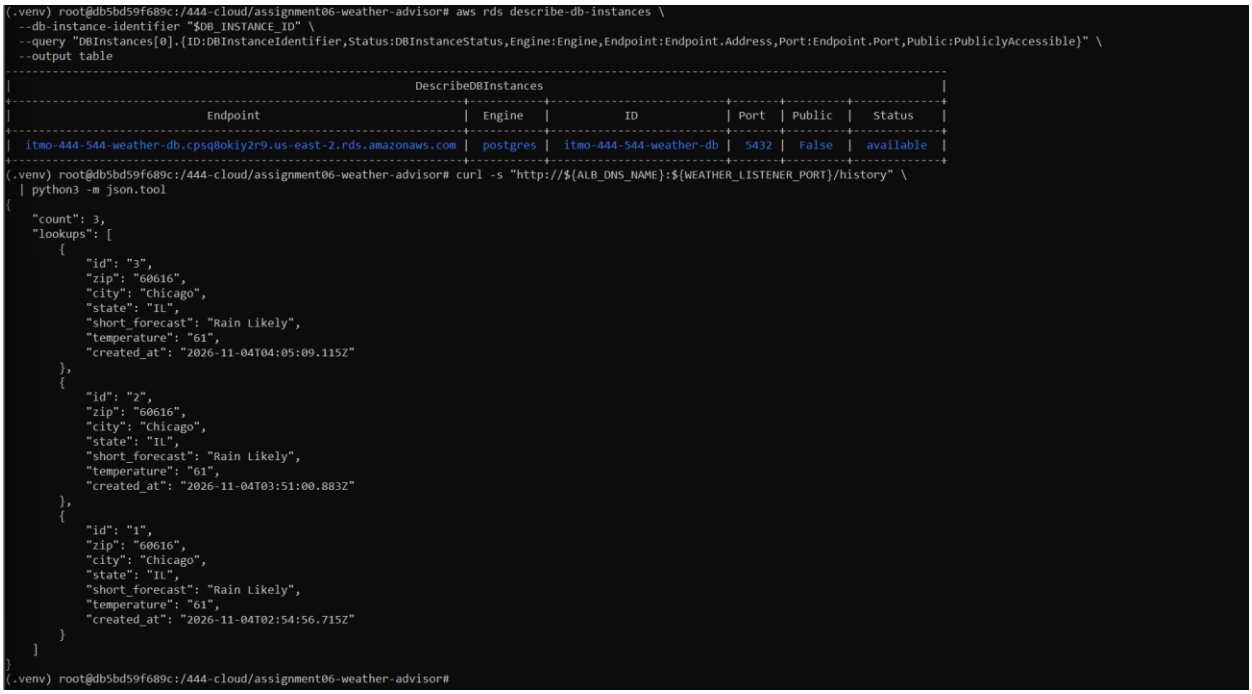
Automated Backups: []

4. Weather Advisor Working: I tested the Advisor application using ZIP code 60616. The Advisor successfully retrieved the weather data and returned a recommendation for the location.
   The application also returned a successful JSON response from the /advice endpoint for the same ZIP code, confirming that the Advisor service was working correctly with the Weather service.

   source = gemini
   model = gemini-3.6-flash
   advice = wear waterproof/wind protection.



5. Weather Logging and Database History: I verified that successful Weather requests were being stored in the PostgreSQL database. The /history endpoint returned previous weather lookups, and the database query showed matching records stored in RDS. This confirmed that the application was using RDS for persistent history instead of keeping the data only on the EC2 instance.



```
(.venv) root@db5bd59f689c:/444-cloud/assignment06-weather-advisor# aws rds describe-db-instances \
  --db-instance-identifier "$DB_INSTANCE_ID" \
  --query "DBInstances[0].{ID:DBInstanceIdentifier,Status:DBInstanceStatus,Engine:Engine,Endpoint:Endpoint.Address,Port:Endpoint.Port,Public:PubliclyAccessible}" \
  --output table
-------------------------------------------------------------------------------------------------------------------
|                                                DescribeDBInstances                                              |
+-----------------------------------------------------------+----------+-------------------------+------+--------+------------+
|                          Endpoint                         |  Engine  |           ID            | Port | Public |   Status   |
+-----------------------------------------------------------+----------+-------------------------+------+--------+------------+
|  itmo-444-544-weather-db.cpsq8okiy2r9.us-east-2.rds.amazonaws.com |  postgres |  itmo-444-544-weather-db |  5432 |  False  |  available  |
+-----------------------------------------------------------+----------+-------------------------+------+--------+------------+
(.venv) root@db5bd59f689c:/444-cloud/assignment06-weather-advisor# curl -s "http://${ALB_DNS_NAME}:${WEATHER_LISTENER_PORT}/history" \
  | python3 -m json.tool
{
    "count": 3,
    "lookups": [
        {
            "id": "3",
            "zip": "60616",
            "city": "Chicago",
            "state": "IL",
            "short_forecast": "Rain Likely",
            "temperature": "61",
            "created_at": "2026-11-04T04:05:09.115Z"
        },
        {
            "id": "2",
            "zip": "60616",
            "city": "Chicago",
            "state": "IL",
            "short_forecast": "Rain Likely",
            "temperature": "61",
            "created_at": "2026-11-04T03:51:00.883Z"
        },
        {
            "id": "1",
            "zip": "60616",
            "city": "Chicago",
            "state": "IL",
            "short_forecast": "Rain Likely",
            "temperature": "61",
            "created_at": "2026-11-04T02:54:56.715Z"
        }
    ]
}
(.venv) root@db5bd59f689c:/444-cloud/assignment06-weather-advisor#
```

6. **Cache Test:** I tested the Weather service twice using the same ZIP code. The first request took longer because the application had to retrieve fresh forecast data, while the second request was much faster and returned a cache hit.
   A short cache helps reduce unnecessary calls to the external weather API and improves response time. However, the cache should not be too long because weather information can become outdated.

```
(.venv) root@db5bd59f689c:/444-cloud/assignment06-weather-advisor# curl -s "http://${ALB_DNS_NAME}:${WEATHER_LISTENER_PORT}/history" \
  | python3 -m json.tool
{
    "count": 3,
    "lookups": [
        {
            "id": "3",
            "zip": "60616",
            "city": "Chicago",
            "state": "IL",
            "short_forecast": "Rain Likely",
            "temperature": "61",
            "created_at": "2026-11-04T04:05:09.115Z"
        },
        {
            "id": "2",
            "zip": "60616",
            "city": "Chicago",
            "state": "IL",
            "short_forecast": "Rain Likely",
            "temperature": "61",
            "created_at": "2026-11-04T03:51:00.883Z"
        },
        {
            "id": "1",
            "zip": "60616",
            "city": "Chicago",
            "state": "IL",
            "short_forecast": "Rain Likely",
            "temperature": "61",
            "created_at": "2026-11-04T02:54:56.715Z"
        }
    ]
}
(.venv) root@db5bd59f689c:/444-cloud/assignment06-weather-advisor# curl -s -o /dev/null -w "First request: %{time_total}s\n" \
  "http://${ALB_DNS_NAME}:${WEATHER_LISTENER_PORT}/weather?zip=60616"
First request: 0.655923s
(.venv) root@db5bd59f689c:/444-cloud/assignment06-weather-advisor# curl -s -o /dev/null -w "Second request: %{time_total}s\n" \
  "http://${ALB_DNS_NAME}:${WEATHER_LISTENER_PORT}/weather?zip=60616"
Second request: 0.105849s
(.venv) root@db5bd59f689c:/444-cloud/assignment06-weather-advisor# curl -s \
  "http://${ALB_DNS_NAME}:${WEATHER_LISTENER_PORT}/weather?zip=60616" \
  | python3 -m json.tool
{
    "zip": "60616",
    "city": "Chicago",
    "state": "IL",
    "forecast": {
        "name": "Today",
        "isDaytime": true,
        "windSpeed": "15 mph",
        "temperature": 64,
        "shortForecast": "Scattered Rain Showers then Patchy Drizzle",
        "temperatureUnit": "F",
        "precipitationChance": 40
    },
    "latitude": "41.8426",
    "longitude": "-87.6306",
    "cache": "hit"
}
(.venv) root@db5bd59f689c:/444-cloud/assignment06-weather-advisor#
```

7. **Least-Privilege IAM Test:** I verified the IAM permissions for both application roles using the policy simulator. The Weather role was allowed to read only the database password parameter, while the Advisor role was allowed to read only the Gemini API key parameter.
   This follows the principle of least privilege because each tier can access only the secret it needs.
   If the Advisor tier were compromised, an attacker could potentially access the Gemini key, but the database password would still remain protected.

```
(.venv) root@db5bd59f689c:/444-cloud/assignment06-weather-advisor# ACCOUNT_ID=$(aws sts get-caller-identity \
  --query Account \
  --output text)

GEMINI_ARN="arn:aws:ssm:${AWS_REGION}:${ACCOUNT_ID}:parameter${GEMINI_PARAM_NAME}"
DB_ARN="arn:aws:ssm:${AWS_REGION}:${ACCOUNT_ID}:parameter${DB_PASSWORD_PARAM}"
(.venv) root@db5bd59f689c:/444-cloud/assignment06-weather-advisor# for ROLE in "$WEATHER_ROLE_NAME" "$ADVISOR_ROLE_NAME"; do
  ROLE_ARN=$(aws iam get-role \
    --role-name "$ROLE" \
    --query Role.Arn \
    --output text)

  for P in "$DB_ARN" "$GEMINI_ARN"; do
    printf "%-40s %-50s " "$ROLE" "${P##*parameter}"

    aws iam simulate-principal-policy \
      --policy-source-arn "$ROLE_ARN" \
      --action-names ssm:GetParameter \
      --resource-arns "$P" \
      --query "EvaluationResults[0].EvalDecision" \
      --output text
  done
done
allowed4-544-weather-role               /itmo-444-544/db-password
implicitDeny-weather-role               /itmo-444-544/gemini-api-key
implicitDeny-advisor-role               /itmo-444-544/db-password
allowed4-544-advisor-role               /itmo-444-544/gemini-api-key
(.venv) root@db5bd59f689c:/444-cloud/assignment06-weather-advisor#
```

8. **Network Security Test:** I verified that the application instance could not be accessed directly from the internet on port 3000, while the same service was reachable successfully through the Application Load Balancer.
   The direct request returned no response, while the request through the ALB returned HTTP 200. This confirms that the application security group only allows traffic from the ALB security group.
   Using another security group as the source is more restrictive than allowing a CIDR range because access is tied to authorized AWS resources instead of a broad IP address range.

```
(.venv) root@db5bd59f689c:/444-cloud/assignment06-weather-advisor# curl -m 5 -s -o /dev/null \
  -w "Direct instance: %{http_code}\n" \
  "http://${WEATHER_IP}:3000/" \
  || echo "Direct instance: no answer (good)"
Direct instance: 000
Direct instance: no answer (good)
(.venv) root@db5bd59f689c:/444-cloud/assignment06-weather-advisor# curl -m 5 -s -o /dev/null \
  -w "Through ALB: %{http_code}\n" \
  "http://${ALB_DNS_NAME}:${WEATHER_LISTENER_PORT}/"
Through ALB: 200
(.venv) root@db5bd59f689c:/444-cloud/assignment06-weather-advisor#
```

9. **Database Failure Test:** I tested how the Weather service behaved when access to the PostgreSQL database was temporarily removed. The application continued serving forecast information even though database-backed history and caching were unavailable. The application logged a database error instead of crashing. The short pause before retrying helps prevent the service from continuously reconnecting to a database that is temporarily unavailable.
   This test showed that the Weather service can continue operating in a degraded mode when the database dependency fails.

```
(.venv) root@db5bd59f689c:/444-cloud/assignment06-weather-advisor# curl -s "http://${ALB_DNS_NAME}:${WEATHER_LISTENER_PORT}/weather?zip=60616" \
  | python3 -c "import sys,json; d=json.load(sys.stdin); print(d['city'], 'cache =', d['cache'])"
Chicago cache = unavailable
(.venv) root@db5bd59f689c:/444-cloud/assignment06-weather-advisor# curl -s -o /dev/null \
  -w "history while database is cut off: %{http_code}\n" \
  "http://${ALB_DNS_NAME}:${WEATHER_LISTENER_PORT}/history"
history while database is cut off: 503
(.venv) root@db5bd59f689c:/444-cloud/assignment06-weather-advisor#
```

10. **Gemini Failure and Fallback:** I tested the Advisor service while the Gemini API was unavailable. Instead of failing completely, the application returned a fallback recommendation and identified the response source as fallback.
    The application also logged the Gemini error so the failure could be diagnosed without exposing sensitive information.
    Clearly identifying fallback responses is important because users should know when the recommendation was generated by the local fallback logic rather than the Gemini model.

```
}
(.venv) root@db5bd59f689c:/444-cloud/assignment06-weather-advisor# sleep 70
(.venv) root@db5bd59f689c:/444-cloud/assignment06-weather-advisor# for i in 1 2 3; do
  curl -s "http://${ALB_DNS_NAME}:${ADVISOR_LISTENER_PORT}/advice?zip=60616" \
    | python3 -c "import sys,json; d=json.load(sys.stdin); print(d['source'], '-', d['advice'])"
done
fallback - Wear a light jacket or a sweater over long sleeves. Bring an umbrella or a waterproof jacket.
fallback - Wear a light jacket or a sweater over long sleeves. Bring an umbrella or a waterproof jacket.
fallback - Wear a light jacket or a sweater over long sleeves. Bring an umbrella or a waterproof jacket.
(.venv) root@db5bd59f689c:/444-cloud/assignment06-weather-advisor#
```

11. **Auto Scaling Recovery:** I terminated the Weather EC2 instance to test whether the Auto Scaling Group would replace it automatically. During the outage, the Advisor returned 502 weather_service_unavailable instead of hanging or crashing.
    The Auto Scaling Group immediately began launching a replacement instance. AWS completed the replacement EC2 launch in about 31 seconds, after which the application needed additional time to finish bootstrapping and pass the load balancer health checks before the Weather service returned HTTP 200 again.
    This test showed that EC2 instances should be treated as replaceable infrastructure. Persistent application state should be stored outside the instance, such as in RDS, so a replacement instance can continue using data created by the previous one.

```
(.venv) root@db5bd59f689c:/444-cloud/assignment06-weather-advisor# aws autoscaling describe-scaling-activities \
  --auto-scaling-group-name "$WEATHER_ASG_NAME" \
  --max-items 10 \
  --query "Activities[].{Description:Description,Start:StartTime,End:EndTime,Status:StatusCode}" \
  --output table
-------------------------------------------------------------------------------------------------------------------
|                                                DescribeScalingActivities                                        |
+-----------------------------------------------+--------------------------+-----------------------------------+-------------+
|                  Description                  |           End            |               Start               |   Status    |
+-----------------------------------------------+--------------------------+-----------------------------------+-------------+
|  Launching a new EC2 instance: i-0b3d18a71a3541dc2 |  2026-11-04T14:36:43+00:00 |  2026-11-04T14:36:26.900000+00:00  |  Successful |
|  Terminating EC2 instance: i-03ebbad59fa35048f     |  2026-11-04T14:41:34+00:00 |  2026-11-04T14:36:12.135000+00:00  |  Successful |
|  Launching a new EC2 instance: i-03ebbad59fa35048f |  2026-11-04T14:32:34+00:00 |  2026-11-04T14:32:29.380000+00:00  |  Successful |
|  Terminating EC2 instance: i-08c0f96e70f597b63     |  2026-11-04T14:37:58+00:00 |  2026-11-04T14:32:16.350000+00:00  |  Successful |
|  Launching a new EC2 instance: i-08c0f96e70f597b63 |  2026-11-04T14:25:57+00:00 |  2026-11-04T14:25:50.504000+00:00  |  Successful |
|  Terminating EC2 instance: i-0c5fe7e4ef1ddb5d5     |  2026-11-04T14:31:26+00:00 |  2026-11-04T14:25:44.027000+00:00  |  Successful |
|  Launching a new EC2 instance: i-0c5fe7e4ef1ddb5d5 |  2026-11-04T13:55:59+00:00 |  2026-11-04T13:55:54.807000+00:00  |  Successful |
|  Terminating EC2 instance: i-06051eb0d3789d72a     |  2026-11-04T14:00:54+00:00 |  2026-11-04T13:55:52.844000+00:00  |  Successful |
|  Launching a new EC2 instance: i-06051eb0d3789d72a |  2026-11-04T13:40:05+00:00 |  2026-11-04T13:40:00.150000+00:00  |  Successful |
|  Terminating EC2 instance: i-02adc56510ab76659     |  2026-11-04T13:45:00+00:00 |  2026-11-04T13:39:58.450000+00:00  |  Successful |
+-----------------------------------------------+--------------------------+-----------------------------------+-------------+
(.venv) root@db5bd59f689c:/444-cloud/assignment06-weather-advisor#
```

12. **Secrets and SSM:** I stored the Gemini API key and the database password in AWS Systems Manager Parameter Store as SecureString values instead of placing them directly in .env, source code, or EC2 user-data.
    Each application role was given permission to read only the secret that it required. The Weather role could access only the database password, while the Advisor role could access only the Gemini API key.
    This approach reduces the risk of exposing credentials through Git, application files, logs, screenshots, or instance configuration.

13. **Backup Decision:** For this lab, I disabled automated backups and final snapshots because the database contained temporary test data and I wanted to avoid leaving unnecessary AWS resources after cleanup.
    The tradeoff is that once the database is deleted, the stored data cannot be recovered. In a real production environment, I would enable automated backups and an appropriate retention period for any data that is important or cannot be easily recreated.

14. **Data Sent to Gemini:** The Advisor sends only the ZIP code and public weather forecast information to Gemini. I did not send database passwords, API keys, user credentials, or private database records.
    Using the free Gemini tier still matters because the request data is being sent to a third-party AI service outside my AWS environment. For a real application, I would review the provider's privacy, retention, and data-use policies before sending any sensitive or user-specific information.

15. Security and Privacy Review:

a. **Secret Storage:** Even when secrets are stored in SSM Parameter Store, they could still be exposed through shell history, process arguments, logs, screenshots, or accidentally committed Git files. Sensitive values should never be printed or stored unnecessarily.

b. **Least Privilege:** If both application roles had ssm:* access to all resources, compromising either service could expose every SSM parameter available to those roles. Restricting each role to one required secret limits the damage of a compromise.

c. **Network Exposure:** The Application Load Balancer is public because users need a public entry point to the application. The EC2 instances and RDS database do not need direct public access because traffic should pass through controlled services and security groups.

d. **Shared Security Group:** Because both application tiers use the same application security group, an Advisor instance receives some network access that it does not actually need. A stronger design would use separate security groups for the Weather and Advisor tiers and allow database access only from the Weather security group.

e. **Database Login:** The application uses the database master account in this lab. In a production system, I would create a dedicated application database user with only the required permissions, such as SELECT and INSERT on the weather lookup table.

f. **Encryption:** TLS certificate verification confirms that the application is communicating with the expected database server. TLS without certificate verification encrypts the traffic, but it does not provide the same protection against connecting to an impersonated server.

g. **SQL Injection:** The application uses parameterized SQL queries instead of directly combining user input with SQL strings. If user input were joined directly into a query, an attacker could potentially change the meaning of the SQL statement and access or modify data.

h. **Backups:** Because backup retention was set to zero and no snapshots were kept, deleting the database permanently removes its data. For a real application, I would enable automated backups and choose an appropriate retention period.

i. **Data Lifecycle:** In a real organization, decisions about when data can be deleted should follow company policy, legal requirements, security requirements, and the decisions of the people responsible for the data.

j. **Untrusted AI Output:** The application escapes AI-generated text before displaying it on the web page. If raw model output were inserted directly into the page, malicious HTML or JavaScript could potentially be executed by a browser.

k. **Data Sent to a Third Party:** Before using Gemini with real users or an organization-owned account, I would review the provider's privacy, retention, security, and data-use policies to make sure the application follows organizational requirements.

l. **Abuse and Cost:** Because the Advisor endpoint is public, someone could repeatedly call it and consume the available Gemini quota. Rate limiting, authentication, AWS WAF rules, and monitoring could be used to reduce abuse.
m. **Availability:** The application clearly identifies degraded responses using values such as source: fallback or cache: unavailable. This is important because it shows users and administrators that a dependency failed instead of making the system appear fully operational.

16. **Graduate Python Requirements:** I completed the infrastructure portion of the assignment using Python and boto3. The project includes create_database.py, delete_database.py, create_weather_stack.py, and delete_weather_stack.py, along with reusable modules inside the stack/ package. The Python code creates and removes the RDS database, security groups, SSM parameters, IAM roles and policies, Application Load Balancer, listeners, target groups, launch templates, and Auto Scaling Groups.
    The create scripts were designed to be idempotent so running them again does not create duplicate resources, while the delete scripts tolerate resources that are already missing.
    Configuration values such as resource names, ports, instance types, database settings, and cache values are read from the environment configuration instead of being hard-coded.
    The final unit-test run completed successfully with: 24 passed, 16 deselected.

17. **What Moto Could and Could Not Prove:** Moto was useful for testing the Python infrastructure code without making real AWS calls. It helped verify resource configuration, IAM policies, security group rules, create/delete behavior, idempotency, and generated launch-template data.
    However, Moto could not prove real AWS networking, actual RDS TLS certificate behavior, real load balancer health checks, real backup behavior, or EC2 replacement timing. Those parts required integration and chaos testing against the deployed AWS environment.

18. **Python vs. Bash:** The Python version was easier to test and maintain because boto3 returns structured data and allows specific AWS exceptions to be handled directly. It also made it easier to reuse functions and verify idempotent behavior with pytest.
    Bash was faster for running individual AWS commands during troubleshooting, but the Python implementation was more organized and easier to test automatically.

19. **Cleanup Verification:** After testing was complete, I deleted the Weather and Advisor stack first and then removed the RDS database and related resources.
    The final AWS verification commands returned empty results for the database instance, snapshots, automated backups, load balancer, Auto Scaling Groups, launch templates, and lab IAM roles. This confirmed that the lab resources had been removed successfully.
    I also deleted the Gemini API key parameter from SSM Parameter Store and verified that the parameter no longer existed.

```
(.venv) root@db5bd59f689c:/444-cloud/assignment06-weather-advisor# aws iam list-roles \
  --query "Roles[?starts_with(RoleName, 'itmo-444-544')].RoleName"
[]
(.venv) root@db5bd59f689c:/444-cloud/assignment06-weather-advisor# aws ssm delete-parameter \
  --name "$GEMINI_PARAM_NAME"
(.venv) root@db5bd59f689c:/444-cloud/assignment06-weather-advisor# aws ssm describe-parameters \
  --query "Parameters[?Name=='${GEMINI_PARAM_NAME}'].Name"
[]
(.venv) root@db5bd59f689c:/444-cloud/assignment06-weather-advisor#
```

20. Github repository